A scaling algorithm to equilibrate both rows and columns norms in matrices

Daniel Ruiz

ENSEEIHT–IRIT, Toulouse, France

Second International Workshop on Combinatorial Scientific Computing,
Toulouse, June 21 - 23th, 2005

Outline

Outline

Scaling Matrices

$$\widehat{\mathbf{A}} = \mathbf{D}_1 \mathbf{A} \mathbf{D}_2 \text{ with } \mathbf{D}_{\{1,2\}} \text{ diagonal matrices}$$

Motivations :

- Equilibration (scaling physical units)
- Balancing (similarity transformations)
- Good pivoting strategy
- Numerical/optimal properties
- Statistical applications (Markov chains, . . .)

HSL scaling routines

MC29 and MC30 (symmetric case) [Curtis and Reid (1972)]

$$\textbf{minimizes} \quad \sum_{a_{ij} \neq 0} (\log |a_{ij}|)^2$$

MC64 [Duff and Koster (1999)]
finds a maximum product transversal and scales it to identity
(can also be used to find a maximum transversal)

Sinkhorn and Knopp (1967) method

alternately normalize rows and columns
and iterate on that

- With the max-norm :
  converges in one step of row and column scaling
- With the one-norm :
  if $\mathbf{A}$ has support (e.g. there exists a full transversal), generates a sequence $\mathbf{A}^{(k)}$ converging to **doubly stochastic** form
- Does not preserve symmetry
- Parallelism is possible

Bunch (1971)

Equilibration of **symmetric** matrices in the **max-norm**

- Operates on the lower triangular part of **A** :
  For $i = 1, 2, \ldots, n$ compute **sequentially**,

$$\delta_i = \max\left\{\sqrt{|a_{ii}|},\ \max_{1\leq j\leq i-1}|\delta_j^{-1}a_{ij}|\right\}$$

  and set $\mathbf{D} = diag(\delta_i^{-1})$
- Permutation dependant (sets the first element a_{11} in **A** to 1)
- Converges in one sweep through the rows of **A**
- Sequential algorithm

Algorithm : Simultanous row and column iterative scaling

Begin

1. Set: $\widehat{\mathbf{A}}^{(0)} = \mathbf{A}, \quad \mathbf{D}_1^{(0)} = \mathbf{I}_m, \quad$ and $\mathbf{D}_2^{(0)} = \mathbf{I}_n$
2. **For** $k = 1, 2, \ldots,$ until convergence **Do:**
   - i. $\mathbf{D}_R = \mathrm{diag}(\sqrt{\|\mathbf{r}_i^{(k)}\|_\star})_{i=1,\ldots,m}, \quad$ and
     $\mathbf{D}_C = \mathrm{diag}(\sqrt{\|\mathbf{c}_j^{(k)}\|_\star})_{j=1,\ldots,n}$
   - ii. $\widehat{\mathbf{A}}^{(k+1)} = \mathbf{D}_R^{-1} \widehat{\mathbf{A}}^{(k)} \mathbf{D}_C^{-1}$
   - iii. $\mathbf{D}_1^{(k+1)} = \mathbf{D}_1^{(k)} \mathbf{D}_R^{-1}, \quad$ and $\mathbf{D}_2^{(k+1)} = \mathbf{D}_2^{(k)} \mathbf{D}_C^{-1}$
3. **EndDo**

End

- $\|\cdot\|_\star$ denotes any of the usual vector norms
- Convergence is obtained when (for a given value of $\varepsilon > 0$)

$$\max_{1\le i\le m} \left\{ |(1 - \|\mathbf{r}_i^{(k)}\|_\star)| \right\} \le \varepsilon \text{ and } \max_{1\le j\le n} \left\{ |(1 - \|\mathbf{c}_j^{(k)}\|_\star)| \right\} \le \varepsilon$$

General properties

- Preserves symmetry
- Permutation independant
- Natural parallelism
- With the max-norm :
  linear convergence with assymtotic rate of 1/2
  works also for rectangular matrices
- With the one-norm :
  same type of results as with the Sinkhorn-Knopp method

With the max-norm

- It is easy to see that, after the first iteration ($k \geq 1$), all entries in $\widehat{\mathbf{A}}^{(k)}$ are less than one in absolute value.
- Consider j_0 such that $|a_{ij_0}^{(k)}| = \|\mathbf{r}_i^{(k)}\|_\infty$ (maximal entry in row i at step (k)). Then :

$$1 \geq |a_{ij_0}^{(k+1)}| = \frac{|a_{ij_0}^{(k)}|}{\sqrt{\|\mathbf{r}_i^{(k)}\|_\infty}\sqrt{\|\mathbf{c}_{j_0}^{(k)}\|_\infty}} = \frac{\sqrt{|a_{ij_0}^{(k)}|}}{\sqrt{\|\mathbf{c}_{j_0}^{(k)}\|_\infty}} \geq \sqrt{|a_{ij_0}^{(k)}|}$$

since $|a_{ij_0}^{(k)}| = \|\mathbf{r}_i^{(k)}\|_\infty$ and $\|\mathbf{c}_{j_0}^{(k)}\|_\infty \leq 1$ for any $k \geq 1$.
Similar result holds for the columns.
- In conclusion, the iterations provide scaled matrices $\widehat{\mathbf{A}}^{(k)}$, $k = 1, 2, \ldots$, with the following properties

$$\forall k \geq 1,\ 1 \leq i \leq m,\ \ \sqrt{\|\mathbf{r}_i^{(k)}\|_\infty} \leq |a_{ij_0}^{(k+1)}| \leq \|\mathbf{r}_i^{(k+1)}\|_\infty \leq 1\ ,$$

and

$$\forall k \geq 1,\ 1 \leq j \leq n,\ \ \sqrt{\|\mathbf{c}_j^{(k)}\|_\infty} \leq |a_{i_0 j}^{(k+1)}| \leq \|\mathbf{c}_j^{(k+1)}\|_\infty \leq 1\ ,$$

which shows that both row and column norms must converge to 1, with an assymptotic rate of $\frac{1}{2}$.

With the one-norm [Parlett and Landis (1982), Ruiz (2001)]

1. If $\mathbf{A}$ has support, then $\mathbf{S} = \lim_{k\to\infty} \widehat{\mathbf{A}}^{(k)}$ exists and is doubly stochastic.
2. If $\mathbf{A}$ has total support, then both $\mathbf{D}_1 = \lim_{k\to\infty} \mathbf{D}_1^{(k)}$ and $\mathbf{D}_2 = \lim_{k\to\infty} \mathbf{D}_2^{(k)}$ exist and $\mathbf{S} = \mathbf{D}_1\mathbf{A}\mathbf{D}_2$.
3. If $\mathbf{A}$ has support but not total support, then for each entry a_{ij} that cannot be permuted onto a full,
$$\lim_{k\to\infty} a_{ij}^{(k)} = 0.$$
4. Finally, in the case of symmetric matrices, the algorithm builds a symmetric path towards this doubly stochastic limit (when it exists).

Extensions

- To any of the ℓ_p norms, $1 \leq p < \infty$ [Rothblum, Schneider, and Schneider (1994)]
  consider the **Hadamar p^{th} power** of **A** (in absolute value), apply the algorithm in the one norm, and take the **Hadamar p^{th} root** of the results.
- To multidimensional matrices
  Example : $\mathbf{A} = (a_{ijk}) \geq 0$, and consider its one dimensional marginals

$$x_i = \sum_{j,k} a_{ijk} \; ; \; y_j = \sum_{i,k} a_{ijk} \; ; \; z_k = \sum_{i,j} a_{ijk} .$$

  Then, set the idea is to scale **A** to $\widehat{\mathbf{A}}$ by setting

$$\hat{a}_{ijk} = \frac{a_{ijk}}{\sqrt[3]{x_i}\sqrt[3]{y_j}\sqrt[3]{z_k}}$$

  for all i, j, k, and to iterate on that.

Scaling to help Numerical pivoting in Gaussian elimination

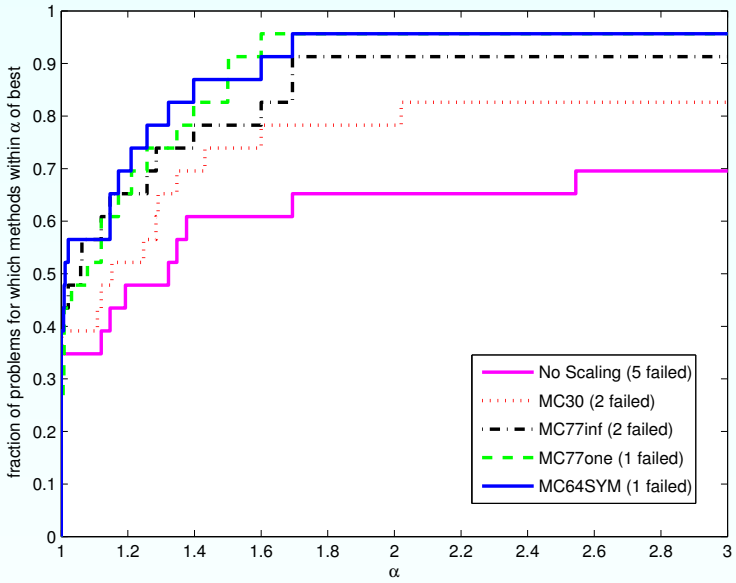
Matrix	NoScaling	MC77inf		MC77one		MC64SYM	
	Facto	Facto	Scal	Facto	Scal	Facto	Scal
Augmented systems (AUGSYS set)							
BLOWEYA	CPU	0.08	0.04	0.09	0.04	0.08	0.03
BRATU3D	PREC	PREC	–	77.6	0.06	PREC	–
NCVXQP1	108.	13.6	0.03	19.3	0.01	12.8	0.43
NCVXQP5	MEM	138.	0.19	140.	0.18	139.	2.94
NCVXQP7	MEM	MEM	–	CPU	–	1307.	21.9
bloweybl	CPU	0.06	0.03	0.08	0.04	0.06	0.03
cvxqp3	263.	30.6	0.03	37.0	0.04	37.0	0.94
stokes128	2.03	0.81	0.21	0.79	0.16	0.81	0.29
stokes64	0.18	0.14	0.04	0.14	0.01	0.13	0.06
tuma2	0.08	0.08	0.01	0.06	0.01	0.04	0.01
TOTAL solved	18/23	21/23		22/23		22/23	

Table: Factorization time (columns Facto) and scaling time (columns Scal). Times in seconds. AMD ordering used. Number of steps for MC77 set to 20. CPU: the maximum CPU time exceeded. MEM: MA57 ran out of memory. PREC: problem in precision of the solution.

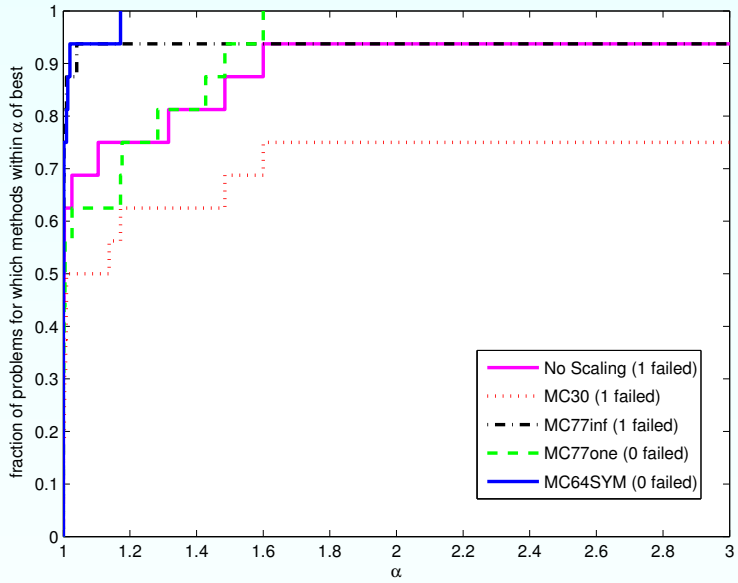
Matrix	NoScaling	MC77inf		MC77one		MC64SYM	
	Facto	Facto	Scal	Facto	Scal	Facto	Scal
General symmetric indefinite (GENSYM set)							
BOYD1	CPU	CPU	–	39.9	0.44	33.9	14.8
c-68	24.5	23.1	0.29	28.5	0.21	22.2	0.13
c-71	76.4	76.2	0.48	108.	0.29	76.4	0.28
vibrobox	1.66	1.26	0.04	1.29	0.06	1.28	0.03
TOTAL solved	15/16	15/16		16/16		16/16	

Table: Factorization time (columns Facto) and scaling time (columns Scal). Times in seconds. AMD ordering used. Number of steps for MC77 set to 20. CPU: the maximum CPU time exceeded. MEM: MA57 ran out of memory. PREC: problem in precision of the solution.

Factorization time with different scalings



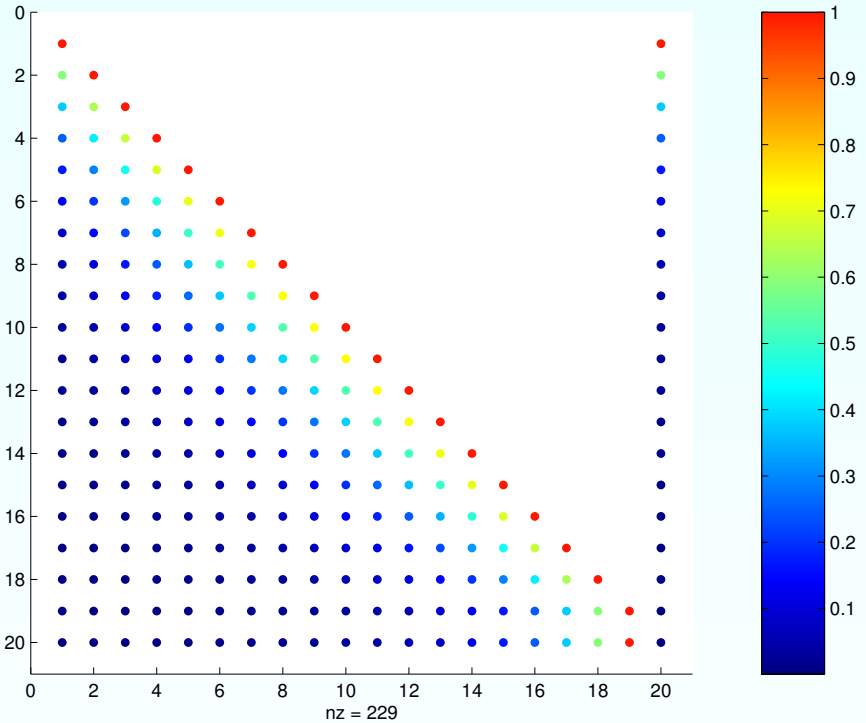
Matrices from the AUGSYS set of sparse matrices



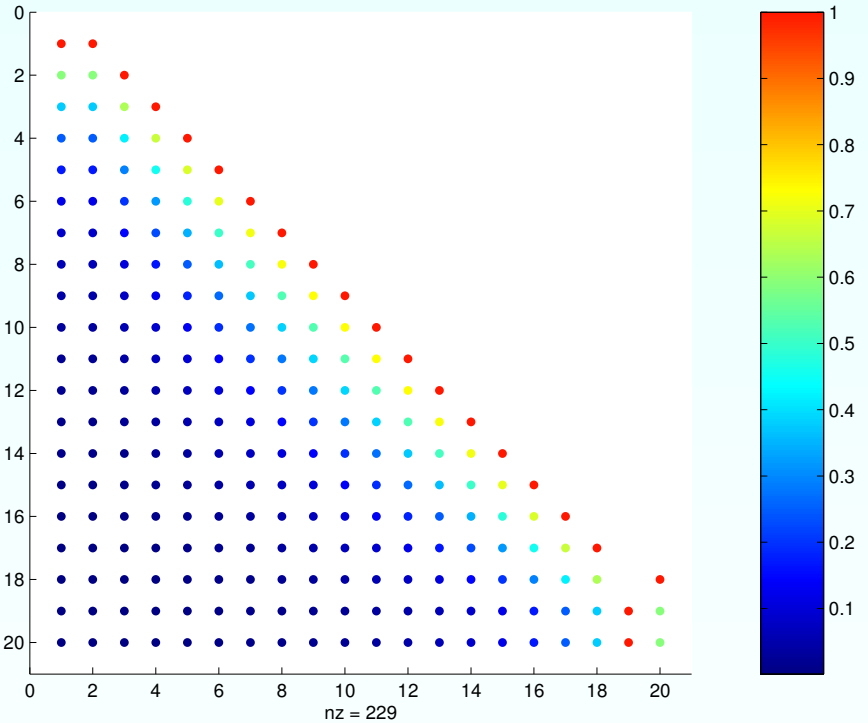
Matrices from the GENSYM set of sparse matrices

Performance profile showing influence of the scaling on the CPU factorization time (AMD ordering)

Scaling plus maximum transversal permutation



Wilkinson matrix after scaling ...



and with maximum transversal permutation

20 steps of scaling in the one-norm followed by 10 steps in infinity norm applied to the Wilkinson matrix, and combined with maximum transversal permutation to determine better pivoting strategy

Scaling with the Block Cimmino method

. . .

Concluding remarks and perspectives

- Incorporate BTF to detect fully-indecomposable sub-components in the matrix, and scale only these components
- Detecting entries without support directly from local numerical behaviour ?
- Acceleration of the scaling algorithm
- Combining Scaling in one-norm + Maximum transversal permutation with dropping strategies and SYMRCM to gather large elements around the diagonal
- Combining Scaling in one or two-norm + Maximum transversal permutation with dropping strategies to build very sparse precondioners
- Code in FORTRAN available in the HSL library (MC77)